Crypto Echo - Concept Questions

Question 1: TCP vs UDP - Why Stateful Communication Matters

Question: This assignment requires you to use TCP instead of UDP. Explain in detail **why TCP is necessary** for this encrypted communication application. In your answer, address: - What specific features of TCP are essential for maintaining encrypted sessions? - What would break or become problematic if we used UDP instead? - How does the stateful nature of TCP support the key exchange and encrypted communication?

Hint: Think about what happens to the encryption keys during a session and what TCP guarantees that UDP doesn't.

TCP allows us to keep a persistent connection with a host or client. This means that we can keep data associated with the other side such as the cryptography keys. If UDP was used, each request would be a new connection. The client would remember the key it needs for the server, but the server wouldn't be able to store information per client unless it had some type of host ip to data mapping which would be extra as opposed to simply maintaining the state using TCP per connection.

Question 2: Protocol Data Unit (PDU) Structure Design

Question: Our protocol uses a fixed-structure PDU with a header containing `msg_type`, `direction`, and `payload_len` fields, followed by a variable-length payload. Explain **why we designed the protocol this way** instead of simpler alternatives. Consider: - Why not just send raw text strings like "ENCRYPT:Hello World"? - What advantages does the binary PDU structure provide? - How does this structure make the protocol more robust and extensible? - What would be the challenges of parsing messages without a structured header?

Hint: Think about different types of data (text, binary, encrypted bytes) and how the receiver knows what it's receiving.

One issue that wouldn't be resolved from TCP is that while a clients bytes will come in order, they will not be viewed as single messages, we can read any number of bytes from our client and it is our job to piece together the messages. If a client sends two messages before we can read them, the TCP client won't tell us they came from two client send()'s. However, if we recv sizeof(header) bytes and then from that grab the payload_len, and then recv(payload_len) we can grab one message, then our next recv will be (sizeof(header)) and we can grab the next message. This cannot be done without that field.

Question 4: Key Exchange Protocol and Session State

Question: The key exchange must happen **before** any encrypted messages can be sent, and keys are **session-specific** (new keys for each connection). Explain **why this design is important** and what problems it solves. Address: - Why can't we just use pre-shared keys (hardcoded in both client and server)? - What security or practical benefits come from generating new keys per session? - What happens if the connection drops after key exchange? Why is this significant? - How does this relate to the choice of TCP over UDP?

Hint: Think about what "session state" means and how it relates to the TCP connection lifecycle.

Each session having a new key means that previous sessions will have different keys from a new session. This means that if someone learns a single key, they can only use it for the current session and all previous and next sessions will still be encrypted and safe, no data will be exposed. If the keys were hardcoded then one exposed key means all past and future data is exposed until the key is changed. If you mean hardcoded as physically hardcoded in the binary (not just a persistent key which can be changed as say an environmnet variable), then if there are two clients, they can both view messages sent from the server by using their own decryption keys for example. So per session key also allows multiple private connections.

Question 5: The Direction Field in the PDU Header

Question: Every PDU includes a `direction` field (DIR_REQUEST or DIR_RESPONSE), even though the client and server already know their roles. Some might argue this field is redundant. Explain **why we include it anyway** and what value it provides. Consider: - How does this field aid in debugging and protocol validation? - What would happen if you accidentally swapped request/response handling code? - How does it make the protocol more self-documenting? - Could this protocol be extended to peer-to-peer communication? How does the direction field help?

Hint: Think about protocol clarity, error detection, and future extensibility beyond simple client-server.

Having a direction field allows us to easily know if the pdu we printed was to the server or from the server. Or in the servers side from the client or to the client. Otherwise, I feel like it wouldn't be too hard to disambiguate with clear variable names and print debugging, but this is a surefire way to have the data more readily viewable for debugging. Having a direction field would help more for an extended scope such as peer to peer communication. A client could then act as a server or a client and maybe be a client and a server at the same time. Printing

and debugging simulatenous messages may be confusing. While the clients would still be connected over TCP and each session will be seperate where one acts a server and the other a client, there are now multiple connections and roles and much more moving parts where this field could help disambiguate things.